Technical Report on the Statistical Pattern Recognition

Computer Assignment #2 Implementation

Code architecture, mathematical foundations, evaluation protocols, and reproducibility analysis

Abstract

This report analyzes the supplied implementation for Statistical Pattern Recognition Computer Assignment #2. The project evaluates two classical distance-based classifiers: Euclidean k-nearest neighbours (kNN) for $k \in \{1, 2, 3\}$ and an assignment-specific minimum-mean-distance (MMD) classifier that assigns each sample to the nearest class centroid. Iris and the dataset labelled "Liquid" are evaluated with leave-one-out prediction, while a two-class Gaussian "Normal" benchmark uses a fixed training/testing partition generated from a fixed random seed. Because the original Assignment 1.2 database files were not included, the implementation uses UCI Iris, UCI Wine as an explicitly labelled Liquid proxy, and a deterministic Gaussian benchmark for the Normal role. The code also includes input validation, deterministic kNN tie handling, reproducible Gaussian generation, CSV and JSON outputs, confusion-matrix figures, and final ZIP packaging.

The report focuses on the computational and mathematical structure rather than on reproducing experimental results. It also notes an important source-format issue: the uploaded file is not a conventional standalone Python module throughout. It contains Colab notebook-style cells, an initial implementation of the core functions, and a later standalone script embedded as a string and copied into the final package. The embedded script is the authoritative executable version used for packaging. This distinction matters for reproducibility and maintenance.

Contents

1 Introduction

The supplied project implements the algorithmic core of a classical statistical pattern-recognition assignment. Its central task is to compare simple non-parametric or prototype-based classifiers under two different evaluation protocols. The first family is k-nearest neighbours, which predicts a sample from the labels of nearby training observations. The second is a minimum-mean-distance classifier, implemented as a nearest-class-centroid rule. Both methods use Euclidean distance directly on the supplied feature coordinates.

The code stays within the requested algorithms. It does not add feature scaling, hyperparameter search, alternative classifiers, learned embeddings, or other statistical decision rules. This keeps the implementation aligned with the assignment rather than turning it into a broader performance-optimization study.

Data provenance is another central issue. The source states that the databases described in Assignment 1.2 were not supplied. Rather than presenting replacements as the original data, the implementation identifies three proxies: UCI Iris for Iris, UCI Wine for Liquid, and a deterministic two-class Gaussian benchmark for Normal. This keeps the limitation clear and avoids overstating what has been reproduced.

The report covers the project from three related perspectives. It describes the classification rules and evaluation procedures, explains the data flow and code organization, and examines how datasets, outputs, metadata, and the standalone implementation are assembled for reproducible use.

2 Project Overview

At a high level, the program follows a simple pipeline. It creates the output directories and fixes the random seed, acquires or reconstructs the three datasets, validates the feature/label inputs, runs kNN for $k = 1, 2, 3$ and the minimum-mean-distance classifier, evaluates predictions with accuracy and confusion matrices, saves machine-readable result tables, and packages the datasets, results, figures, metadata, README, and standalone Python implementation into a ZIP archive.

The source has two overlapping layers. The first resembles an exported Colab notebook, with Markdown strings, a package-installation shell command, repeated imports, and cells that call the experiment functions. The second is a standalone script stored in the triple-quoted string `script_text`. The packaging stage writes that string to `statistical_pattern_recognition_assignment2.py`. The file therefore serves both as a notebook export and as a source for the final standalone script.

This also explains why the uploaded file is not valid standalone Python as a whole: the notebook portion contains a Colab shell command beginning with `!pip`. The embedded standalone script is valid Python and provides a conventional `main()` entry point.

Component	Role	Implementation
Iris	Multiclass benchmark	UCI Iris, downloaded from the public URL with a scikit-learn fallback
Liquid	Assignment proxy	UCI Wine, explicitly labelled as a Liquid proxy
Normal	Controlled synthetic benchmark	Two Gaussian classes, deterministic seed, fixed train/test split
kNN	Local classifier	Euclidean nearest neighbours with $k = 1, 2, 3$ and deterministic ties
MMD	Prototype classifier	Nearest class centroid under Euclidean distance
Evaluation	Protocol-specific assessment	Leave-one-out for Iris/Liquid; fixed testing set for Normal
Outputs	Reproducibility artifacts	CSV tables, JSON metadata, PNG confusion matrices, standalone script, ZIP package

Table 1: Major components identified directly from the supplied implementation.

3 Technical Foundations

3.1 Distance-based classification

The implementation is built around Euclidean distance in a d-dimensional feature space. For two feature vectors $x, z \in \mathbb{R}^d$, the code computes

$$d(x, z) = \|x - z\|_2 = \sqrt{\sum_{j=1}^{d} (x_j - z_j)^2}. \tag{1}$$

Because the classifier uses Euclidean distance directly, the numerical scale of each feature affects the decision. A feature with a larger range can contribute more to the distance than one with a smaller range. The source explicitly avoids standardization and normalization. This is part of the stated methodology, although it has practical consequences for the Wine-based Liquid proxy, whose features are on different scales.

3.2 k-nearest neighbours

Given a labelled training set $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{N}$ and a query point x, the code selects the k training observations with the smallest Euclidean distances to x. Let $\mathcal{N}_k(x)$ denote that neighbour set. The predicted class is the majority class among these neighbours:

$$\hat{y}(x) = \arg\max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(x)} \mathbf{1}\{y_i = c\}. \tag{2}$$

The program evaluates exactly three values of k: 1, 2, and 3. This matches the assignment and avoids adding a separate model-selection procedure.

The implementation also defines a deterministic tie rule. Distances are sorted stably, and when several classes receive the same number of votes, the class of the nearest tied neighbour is selected. This makes the classifier fully specified even when an even value of k produces a tied vote.

3.3 Minimum-mean-distance classifier

The source uses the name “MMD” for minimum-mean-distance. This should not be confused with the separate statistical-learning term “maximum mean discrepancy”, which is also commonly abbreviated MMD. In this project, MMD is a nearest-class-centroid rule.

For class c, let $\mathcal{I}_c = \{i : y_i = c\}$ and let $N_c = |\mathcal{I}_c|$. The class mean is

$$\boldsymbol{\mu}_c = \frac{1}{N_c} \sum_{i \in \mathcal{I}_c} x_i. \tag{3}$$

A query x is assigned to the class whose centroid is closest:

$$\hat{y}(x) = \arg\min_{c \in \mathcal{C}} \|x - \boldsymbol{\mu}_c\|_2 . \tag{4}$$

The method represents each class by a single prototype vector. Compared with kNN, this makes a stronger structural assumption because the class centroid is used as the representative point for the decision. The code does not estimate class covariance matrices and does not implement a quadratic or Mahalanobis decision boundary.

3.4 Accuracy and confusion matrices

For true labels y_i and predictions $\hat{y}_i$, accuracy is

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{y_i = \hat{y}_i\}. \tag{5}$$

The implementation uses scikit-learn's `accuracy_score` and `confusion_matrix`. The confusion matrix uses an explicit label order derived from the union of the true and predicted labels, so the output ordering is deterministic for a given evaluation.

4 Data and Input Processing

4.1 Iris acquisition

The Iris loader first tries to download the UCI ZIP archive from the URL in the source. The archive is stored under the dataset directory and extracted only when the extraction directory does not already exist. The loader then searches for `iris.data`, reads the table without a header, treats question marks as missing values, removes incomplete rows, and renames the five columns as four features and one label.

If the download or parsing step raises an exception, the implementation falls back to scikit-learn's bundled Iris copy. The source describes this as a local fallback for cases in which network access is unavailable. The resulting data are saved as a normalized CSV file and converted to a floating-point feature matrix and a string label vector.

4.2 Liquid proxy acquisition

The Liquid loader follows the same public-download strategy but expects `wine.data`. Since the source explicitly states that the original Liquid database was unavailable, the UCI Wine dataset is stored under the name `liquid_dataset_proxy_wine.csv`. Its first column is treated as the class label and the remaining columns as numerical features. Labels are converted to strings.

The embedded standalone implementation uses scikit-learn's Wine copy as a fallback and places `label` first, followed by the feature columns. The earlier notebook portion contains a less direct fallback with a dead conditional and a later reassignment. That code does not affect the packaged standalone script, but it is a maintenance issue because the file contains two overlapping implementations.

4.3 Normal synthetic benchmark

The Normal dataset is generated rather than downloaded. The implementation fixes the random seed at 42, creates two classes, and samples training and testing observations independently from class-specific multivariate Gaussian distributions. Each class contributes 60 training samples and 40 testing samples.

The class means are

$$\boldsymbol{\mu}_0 = \begin{bmatrix} -1.5 \\ -1.0 \end{bmatrix}, \qquad \boldsymbol{\mu}_1 = \begin{bmatrix} 1.5 \\ 1.0 \end{bmatrix}, \tag{6}$$

and both classes use the covariance matrix

$$\boldsymbol{\Sigma} = \begin{bmatrix} 0.75 & 0.20 \\ 0.20 & 0.75 \end{bmatrix}. \tag{7}$$

Thus, for class c, the generated observations follow the model

$$X \mid Y = c \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}). \tag{8}$$

The covariance matrix is symmetric, and its non-zero off-diagonal entries introduce correlation between the two dimensions. The fixed sample counts and random-generator state make the synthetic benchmark deterministic.

4.4 Input validation

Before classification, `_validate_xy` checks that X is two-dimensional, y is one-dimensional with the same number of samples, the dataset is non-empty, and all entries of X are finite. These checks provide a basic input contract before distance calculations begin.

The validation is intentionally lightweight. It does not check the test feature dimension against the training dimension, verify finiteness of test inputs, or require a specific label dtype. These omissions are acceptable for the controlled experiment paths but make the prediction functions less robust as general-purpose utilities.

5 System Architecture and Code Organization

5.1 Dependencies and configuration

The implementation relies on common scientific Python libraries. NumPy handles numerical arrays, distance calculations, random sampling, stacking, and finite-value checks. pandas handles dataset parsing and CSV serialization. scikit-learn provides the Iris/Wine fallbacks and the accuracy and confusion-matrix utilities. matplotlib and seaborn are used for confusion-matrix figures. The standard library covers URL access, ZIP handling, JSON serialization, path management, and file copying.

The configuration layer defines one seed, an output root, and three subdirectories: `datasets`, `results`, and `figures`. These directories are created with `parents=True, exist_ok=True`, so rerunning the program does not fail because the directories already exist.

5.2 Data acquisition functions

The functions `download_bytes` and `download_uci_zip` separate download handling from dataset parsing. The first creates an HTTP request with a user-agent string and a 60-second

timeout. The second stores the ZIP archive, extracts it into a directory derived from the archive name, and returns that path. The separation keeps the dataset loaders focused on schema handling rather than networking.

5.3 Classifier functions

The prediction layer has three main functions. `knn_predict` computes neighbour distances and majority votes, `mmd_predict` computes one centroid per class and selects the nearest centroid, and `leave_one_out_predictor` applies either classifier in a repeated train-minus-one procedure. This keeps the classifier logic separate from the evaluation protocol.

This separation is useful for leave-one-out evaluation. The classifier functions only receive training and test data; the evaluation wrapper constructs each leave-one-out partition and calls the classifier. The code therefore keeps the prediction rule separate from the evaluation protocol.

5.4 Evaluation and artifact functions

`evaluate_predictions` returns an accuracy value and a confusion matrix. `save_confusion_matrix` writes the matrix as a high-resolution PNG. `run_loo_experiment` evaluates kNN for the three required k values, runs MMD once, and saves the corresponding confusion matrices. `run_holdout_experiment` performs the same type of evaluation using the fixed Normal training and testing partitions.

Finally, `save_results` creates three CSV views: the full performance table, the best row for each dataset, and mean accuracy grouped by dataset and classifier. The `main` function runs the experiments and writes JSON metadata containing the seed, evaluation requirements, and dataset-provenance limitation.

6 Detailed Methodology

6.1 kNN implementation logic

For each test sample, the classifier computes a vector of distances from that sample to every training observation using

```
distances = np.linalg.norm(X_train - sample, axis=1)
nearest_indices = np.argsort(distances, kind="stable")[:k]
```

The first line relies on NumPy broadcasting. If X_{train} has shape (N, d) and the query has shape $(d,)$, subtraction produces an (N, d) matrix of coordinate differences. The Euclidean norm along axis 1 then gives N distances. The second line fully sorts those distances and keeps the first k indices.

A full sort is simple and deterministic, but it is not the most efficient way to select the k smallest distances. A partial-selection method could avoid sorting all N values when $k \ll N$. The current implementation favors straightforwardness and deterministic ordering.

The vote count is stored in a dictionary keyed by class. The implementation finds the largest vote count and, when there is a tie, selects the class of the nearest tied neighbour. Because the distances are sorted stably, this rule is deterministic.

6.2 MMD implementation logic

The centroid classifier first collects unique labels and computes the sample mean for each class:

```
class_means = {
    label: X_train[y_train == label].mean(axis=0)
    for label in classes
}
```

The class means are then stacked into a matrix. For each query, the code computes its Euclidean distance to every class mean and returns the label with the smallest distance. There is no iterative optimization; the only estimated quantities are the class means, obtained directly from the training data.

6.3 Leave-one-out evaluation

For each index i, the evaluation wrapper creates a Boolean mask of length N, excludes position i, and uses the remaining $N-1$ observations for training. The held-out sample is passed as a one-row test array. For dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$, the training subset for observation i is

$$\mathcal{D}_{-i} = \mathcal{D} \setminus \{(x_i, y_i)\}. \tag{9}$$

The classifier is then evaluated on x_i using only $\mathcal{D}_{-i}$. Aggregating these predictions gives the leave-one-out accuracy

$$\mathrm{Acc}_{\mathrm{LOO}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{\hat{y}_{-i}(x_i) = y_i\}. \tag{10}$$

This construction keeps the held-out sample out of the training set. That matters especially for $k = 1$, since including the sample would create an artificial self-neighbour at zero distance.

6.4 Fixed train/test evaluation for Normal

The Normal benchmark uses a different protocol. The generated training set remains fixed, and every test observation is classified using only that set. There is no resampling, cross-validation, or model tuning.

7 Mathematical Formulation

The classifier layer consists of two decision rules based on the same Euclidean distance.

For kNN,

$$\hat{y}_{\mathrm{kNN}}(x) = \arg\max_{c \in \mathcal{C}} \sum_{i \in \mathcal{N}_k(x)} \mathbf{1}\{y_i = c\}. \tag{11}$$

For minimum-mean-distance,

$$\hat{y}_{\mathrm{MMD}}(x) = \arg\min_{c \in \mathcal{C}} d(x, \boldsymbol{\mu}_c), \qquad \boldsymbol{\mu}_c = \frac{1}{N_c} \sum_{i:y_i=c} x_i. \tag{12}$$

The two rules differ in how they represent the training data at prediction time. kNN retains all observations and makes a local decision around each query, whereas MMD represents each class with a single mean vector. The assignment therefore compares a local instance-based rule with a centroid-based rule.

For the synthetic Normal benchmark, the data are generated from multivariate Gaussian class-conditional distributions. However, the classifier does not use the Gaussian density. It classifies the generated samples with kNN and MMD, so it should not be described as a Gaussian likelihood classifier.

8 Optimization and Learning Procedure

There is no gradient-based optimization in this project. Here, "learning" means estimating class means for MMD or retaining the training observations for kNN.

For kNN, there is no fitted parameter vector. The training data themselves constitute the model state. Prediction requires retaining the observations and their labels, followed by distance calculations at inference time.

For MMD, each class mean is estimated from training data using the arithmetic sample mean. This is a closed-form estimator. The code does not perform iterative updates, choose a learning rate, or apply regularization. In symbolic terms, the centroids are determined directly from the supplied training set:

$$\boldsymbol{\mu}_c^{\text{train}} = \frac{1}{N_c} \sum_{i:y_i=c} x_i. \tag{13}$$

Because Iris and Liquid use leave-one-out evaluation, the centroid is recomputed for each held-out sample instead of being estimated once from the full dataset. This keeps the held-out observation out of the training calculation.

The Normal benchmark also has no hyperparameter optimization. The three k values are specified by the assignment rather than selected with a validation set. The results therefore do not support a claim that one of these values was chosen because it generalized best under independent tuning.

9 Implementation Details

9.1 Type hints and data representation

The core functions use NumPy arrays and type annotations such as `np.ndarray`, `Path`, and tuple return types. Labels are converted to one-dimensional string arrays, and feature matrices are stored as floating-point NumPy arrays. After loading, the classifier functions no longer depend on pandas.

9.2 Filesystem design

All generated files are placed beneath one root directory. The layout is stable enough to support automation and packaging:

```
assignment2_outputs/
  datasets/
  results/
  figures/
  final_package/
  statistical_pattern_recognition_assignment2_submission.zip
```

The final package contains copies of the generated datasets, results, figures, and standalone implementation. A README file describes the assignment protocol and the proxy-data limitation. This separates working outputs from the package prepared for submission.

9.3 Reproducible metadata

The JSON metadata records the seed and the evaluation requirements. It also records the key provenance caveat that the original Assignment 1.2 files were not supplied. This information would be easy to lose if only the CSV results were shared.

9.4 Confusion-matrix generation

Confusion matrices are saved at 300 dpi with explicit axis labels and tight bounding boxes. Plotting is separate from prediction and evaluation, so the classifier logic does not depend on visualization.

9.5 Packaging behavior

The packaging block copies the generated directories into `final_package`, writes the standalone source and README, and traverses the package tree to create a ZIP archive. It then attempts to use `google.colab.files.download`. If the Colab API is unavailable, the exception is caught and the ZIP remains on disk.

The packaging code therefore separates artifact creation from browser-based downloading. The latter is environment-specific and is treated as optional.

10 Execution Workflow

The standalone program is driven by `main()`. Its execution sequence is:

1. Load the Iris dataset, using the public download or the local scikit-learn fallback.
2. Load the Wine-based Liquid proxy under the same fallback philosophy.
3. Generate the Normal training and testing partitions from the fixed Gaussian specification.
4. Run the complete leave-one-out experiment on Iris.
5. Run the complete leave-one-out experiment on Liquid.
6. Run the fixed-testing-set experiment on Normal.
7. Concatenate the resulting experiment tables.
8. Save the performance tables and metadata.
9. Print a compact textual completion report and the output location.

The notebook wrapper follows the same overall sequence, while the embedded standalone script repeats the computational definitions and exposes them through `main()`. The duplication does not change the method, but it increases maintenance cost because the two copies can diverge.

11 Design Decisions and Rationale

11.1 Using explicit proxy datasets

A central design choice is the transparent treatment of missing source data. The implementation does not present the replacement datasets as the original databases. It labels Wine as a Liquid proxy and the Gaussian construction as a Normal benchmark. This avoids implying an exact reproduction that the available data do not support.

11.2 Avoiding feature scaling

The source specifies Euclidean distance on the original feature scale and does not introduce feature scaling. This follows the requested methodology but makes the classifier sensitive to feature magnitude. The implementation is therefore best viewed as a direct distance-based baseline rather than a normalized production classifier.

11.3 Stable and deterministic tie handling

Stable sorting and nearest-tied-neighbour resolution make the kNN behavior deterministic. Without an explicit tie policy, tied cases could depend on implementation details.

11.4 Separation of prediction and evaluation

The classifier functions do not need to know whether they are being used for leave-one-out or holdout evaluation. The same prediction logic can therefore be reused under both protocols, while the wrapper functions handle the protocol-specific work.

12 Computational Considerations

Let N denote the number of training observations, d the feature dimension, k the number of neighbours, and C the number of classes.

For one kNN prediction, computing the distances requires $O(Nd)$ arithmetic. The implementation then fully sorts the N distances, adding $O(N \log N)$ comparisons. Thus, for one test point,

$$T_{\text{kNN}} = O(Nd + N \log N). \tag{14}$$

If N_{test} independent test samples are classified, the corresponding work is approximately

$$O\left(N_{\text{test}}(Nd + N \log N)\right). \tag{15}$$

Leave-one-out repeats this process N times, giving approximately

$$O(N^2 d + N^2 \log N). \tag{16}$$

MMD has a different cost profile. Computing all class centroids costs $O(Nd)$ for a fixed training set, and a query requires $O(Cd)$ work to compare it with the class centroids. Therefore, for a fixed train/test partition,

$$T_{\text{MMD}} = O(Nd + N_{\text{test}} Cd). \tag{17}$$

Under leave-one-out, the centroids are recomputed for each held-out observation, giving an overall scaling of about $O(N^2 d)$ for fixed C. The datasets in this assignment are small, so the simple implementation is adequate. The difference in scaling becomes relevant only for larger datasets.

Memory use is modest. The main state is the feature matrix and labels, about $O(Nd)$, plus temporary distance arrays and result tables. The generated PNG figures may occupy more disk space than the numerical arrays, but they are outputs rather than classifier state.

13 Reproducibility and Reliability

The implementation uses several reproducibility measures. The synthetic Normal dataset uses a fixed NumPy generator seed. Downloaded archives are kept in the dataset directory, so repeated runs can reuse existing files. The prediction functions are deterministic, including stable sorting for kNN. Results are saved as CSV files, and the run configuration is recorded in JSON metadata.

The code can also continue with the Iris and Wine proxy experiments when network access fails, provided scikit-learn is available. The fallback path is reported in the console output, which makes the behavior visible in offline or restricted environments.

Reproducibility is not absolute. The source does not pin Python or package versions, and it does not control low-level numerical differences across hardware. Public download availability can also change, although local fallbacks are provided. Finally, the file contains both notebook-facing code and an embedded standalone script, so the two copies can diverge if only one is edited.

14 Limitations and Technical Considerations

14.1 Missing original Assignment 1.2 databases

The main limitation is that the original databases referenced by Assignment 1.2 are not included in the supplied material. UCI Iris, UCI Wine, and the synthetic Gaussian benchmark are clearly identified and reproducible, but they do not establish exact reproduction of the original experiments. Performance should therefore be interpreted only for the datasets actually used by the implementation.

14.2 Feature-scale sensitivity

Because distance is computed directly from the supplied feature coordinates, the geometry changes with feature scale. When features have different units or ranges, some coordinates can dominate the Euclidean distance. This is a property of the chosen method, not a numerical bug.

14.3 Small predefined k set

Only $k = 1, 2, 3$ are evaluated. The implementation does not test a wider range of neighbourhood sizes, use a validation split, or select k through hyperparameter tuning. The comparison is therefore an assignment-defined check over three values rather than a systematic kNN model-selection study.

14.4 Validation scope

The validator checks the training feature matrix and label vector but leaves some possible shape mismatches to NumPy's runtime behavior. A general-purpose interface would also check test dimensionality and finiteness. The current checks are sufficient for the controlled experiment paths but are not a complete defensive interface.

14.5 Source duplication

The uploaded file contains duplicated implementation logic. One copy appears in the notebook-like cells, while the other is stored in `script_text` and written out as the standalone source. This makes the project harder to maintain and allows the two versions to diverge. A cleaner design would use one Python module and import it from the notebook.

14.6 Packaging versus core experiment

The plotting and packaging code is useful for the assignment deliverables, but it is separate from the statistical decision rules. For a research software repository, keeping the core experiment code separate from reporting and packaging utilities would make testing and reuse easier.

15 Overall Technical Assessment

The implementation is technically coherent for a controlled classical pattern-recognition assignment. Its main strengths are the direct implementation of the required classifiers, explicit handling of missing datasets, separation of classifier logic from evaluation, deterministic synthetic-data generation, and structured artifact packaging.

The kNN implementation is simple and fully specified, including input validation and deterministic tie handling. MMD estimates one centroid per class and uses the nearest-centroid rule without iterative optimization. The leave-one-out wrapper excludes the held-out observation from training for every prediction, and the fixed-testing-set path keeps the synthetic test partition separate from training.

From a software-engineering perspective, the main weakness is duplicated code. The file acts as both a notebook export and a generator of a second standalone script, which is less maintainable than a single-source design. The lack of dependency pinning and more extensive interface validation also limits portability.

From a research-methodology perspective, the main interpretive constraint is the use of proxy data. The code states this limitation clearly, but the resulting experiments should not be presented as exact reproductions of the unavailable Assignment 1.2 datasets.

Overall, the project shows practical research-software skills: translating algorithmic requirements into executable procedures, implementing classical classifiers directly, defining evaluation protocols, handling reproducibility constraints, and producing structured artifacts. Its value is mainly as evidence of careful implementation and experimental discipline rather than as a contribution of novel machine-learning methodology.

16 Conclusion

The supplied project implements a reproducible workflow for a classical statistical pattern-recognition assignment. Its core consists of Euclidean kNN for $k = 1, 2, 3$ and a minimum-mean-distance nearest-centroid classifier. Iris and the explicitly labelled Liquid proxy use leave-one-out evaluation, while the Normal benchmark uses a deterministic Gaussian train/test split.

The source combines data acquisition, validation, classification, evaluation, artifact generation, and packaging in one project. The mathematical core is straightforward: Euclidean distance defines neighbourhoods and centroid proximity, arithmetic means define the class prototypes, and accuracy compares predictions with the true labels. The synthetic benchmark also defines the data-generation process through a two-class multivariate Gaussian model.

The project is best understood as a transparent implementation of a defined assignment protocol. Its main limitations are the unavailable original databases, sensitivity to feature scale, the restricted set of predefined k values, limited defensive validation, and duplicated notebook/standalone code. These limitations define the appropriate scope for interpreting the implementation.

For a GitHub-based academic portfolio, the project provides evidence of practical statistical-learning implementation, careful evaluation, and attention to reproducible artifacts. Its main strength is fidelity: the code makes its assumptions and dataset substitutions explicit rather than presenting them as something they are not.